

Git & GitHub Workshop

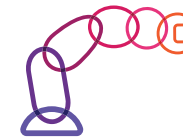
For Linux Users

12 & 14 May 2026



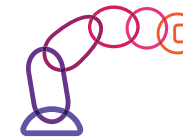
The Workshop

- The workshop consists of two parts.
 - Each part lasts 2 hours 30 minutes with a 10 minute comfort break roughly halfway.
 - The general approach is that there will be 5 to 10 minutes of presentation followed by practical exercises.
- The objectives of this workshop are:
 - To understand what Git is, why it was developed and the basic principles of how it works.
 - To be able to use Git & GitHub when working on your own.
 - To be understand how to work in a small team.
 - To be able to contribute to an Open Source project using GitHub.
 - To understand what GitHub Enterprise is and how you can use it at work.



What is a VCS?

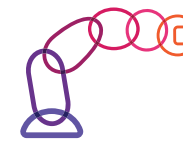
- Git is a Version Control System (VCS)
- A VCS allows you to store different versions of files. This is how a person who does not have a version control system might work on a program.
 - 1) Create the program file and call it "project.py"
 - 2) Spend a few hours coding and you have your first version so you call it "project-draft.py"
 - 3) You add a new feature and save it in "project-draft-2.py"
 - 4) You tidy it up and send it off for review calling it "project-review.py"
 - 5) You get lots of useful feedback and implement it in "project-post-review.py"
 - 6) You send it for review again. Only a few changes this time implemented in "project-final.py"
- This is a manual way of creating revisions. Let's look at the same example but using a VCS.



What is a VCS? - 2

- This is the equivalent of the previous example using a VCS.
 - 1) You create the project in the VCS and add the file called “project.py”.
 - 2) Add some code to the file and test it. Once it works, you add the new revision of “project.py” to the VCS with a message “Implemented first feature. Working”
 - 3) Add some more code for the second feature and add the new revision of “project.py” to the VCS with a message “Implemented second feature. Working”
 - 4) You check your work, write the documentation and add the new revision of “project.py” to the VCS with a message “Added documentation. Ready for review”.
 - 5) After the review, you fix up the issues and add the new revision of “project.py” to the VCS with a message “Fixed review comments. Ready for second review”.
 - 6) The review comes back with a couple of typos to fix. Add the new revision of “project.py” to the VCS with a message “Fixed more review comments. Ready for deployment”.

Using a VCS, the file name does not change. Each revision has its own message to explain the changes made between the labelled revision and the previous one.

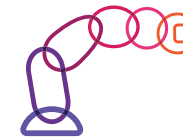


What is a VCS? - 3

- So what about a project that has multiple files? Here is an example:

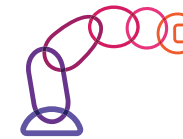
Version 1 "Wrote requirements. Added main.cpp"	my-project/v1/docs/requirements.md /src/main.cpp
Revision 2 "Added camera interface."	my-project/v2/docs/requirements.md /include/camera.h /src/main.cpp /camera.cpp
Revision 3 "Added timer to automatically take photos at timed intervals. Internal defaults to 1 photo per second."	my-project/v3/docs/requirements.md /tests.md /include/camera.h /timer.h /src/main.cpp /camera.cpp /timer.cpp

- As you can see, the messages that go with each revision are much easier to understand than a file name.
- Over the years, there have been many different VCSs, but currently, Git is the most popular.



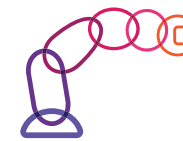
History of Git Development

- Back in 2005, Linus Torvalds was leading the development of the Linux kernel and he needed a replacement for the BitKeeper VCS they had been using. Git was developed with four guiding principles in mind:
 - Patching should take no more than three seconds.
 - Take the Concurrent Versions System (CVS) as an example of what not to do; if in doubt, make the exact opposite decision.
 - Support a distributed, BitKeeper-like workflow.
 - Include very strong safeguards against corruption, either accidental or malicious.
- It was developed very rapidly, starting on the 3 April 2005 with it being good enough to manage the kernel 2.6.12 release on 16th June 2005.
- Torvalds quipped about the name git (which is British English slang for an unpleasant or silly person): "I'm an egotistical bastard, and I name all my projects after myself. First 'Linux', now 'git'."
- The trademark "Git" is registered by the Software Freedom Conservancy.



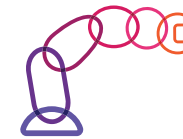
How Git Works

- Git is very fast and efficient and was built rapidly. How does it work?
 - Built on two existing Unix utilities, *diff* and *patch*.
 - Using *diff* and *patch*, you can compare two versions of the same file and generate a patch file. Then you can apply the patch file to the older version to create the newer version, or you can take the new version, reverse the patch file contents and then apply that to get the previous version.
 - DEMO TIME! You don't have to remember this but it is useful background.
 - This is where the phrase “patching a program” comes.
 - This is also the basic principle that Git is built upon.
 - The Git command line interface (CLI) add methods to track each version and methods to create, merge and delete branches (more on those later).



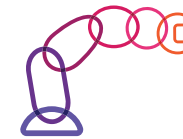
Using Git on your own

- As our first step, I am going to talk you through creating your first GitHub repo and open it in a Codespace. Follow along with me.
 - 1) Login to your GitHub.
 - 2) Create a new repository (repo).
 - 3) Available options are:
 - 1) Public/private – up to you.
 - 2) Templates (ignore as mainly for businesses), select No.
 - 3) README (useful but can be added later) – Enable.
 - 4) .gitignore (Will explain this later) – select No.
 - 5) license (good to do if you creating a public repo) - select MIT License.
 - 6) Ignore copilot.
 - 7) Press “Create repository” *



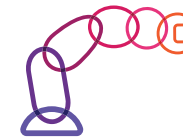
Using Git on your own - 2

- Now to open the new repo in a Codespace.
 - 1) Find your new repo.
 - 2) Click on the green “Code” button.
 - 3) Select the Codespace tab.
 - 4) Once it has started, you should see an editor (a browser version of VSCode).
 - 5) Take a look at the **.git** directory. The **config** file contains information about the *origin* repo (normally the one on GitHub) and any branches.
 - 6) Basic information commands:
 - 1) git status** – Tells you information about the state of the repo.
 - 2) git log** – Tells you about each of the commits in the repo.
 - 7) Update the README file with a line or two of text: **README.md**



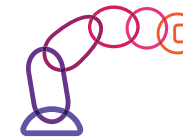
Using Git on your own - 3

- Continued:
 - 1) See what has changed: **git status**
 - 2) Tell Git about the new file by adding it to the repo: **git add README.md**
 - 3) See what has changed: **git status**. The new file is “staged” or ready to be committed to a new revision. You can add many files to the staged before committing.
 - 4) Let's commit the change: **git commit -m “Updated README”**
 - 5) See what has changed: **git status**
 - 6) Check the log to see the new commit: **git log**
- This is the basic



Using Git on your own - 3

- So far, we have made a change in our Codespace (a local change) but the GitHub repo has not been updated.
 - 1) Let's add a new file, **design.md**, and add a couple of lines of text to it.
 - 2) Commit it.
 - 3) Now we want to update the GitHub repo with our two changes so we don't lose them.
 - 4) To do this, we need to "push" our changes to the server using *git push*. The very handy defaults save us having to type the full command *git push origin master*.
 - 5) Check GitHub to see if your changes have appeared.
 - 6) Congratulations. Your code is now backed up and can be shared with others.

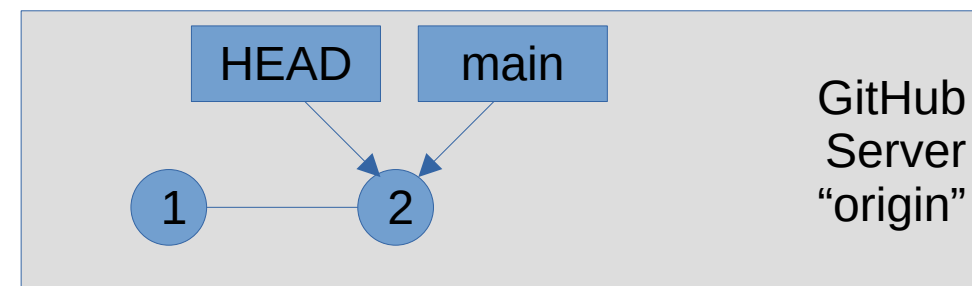
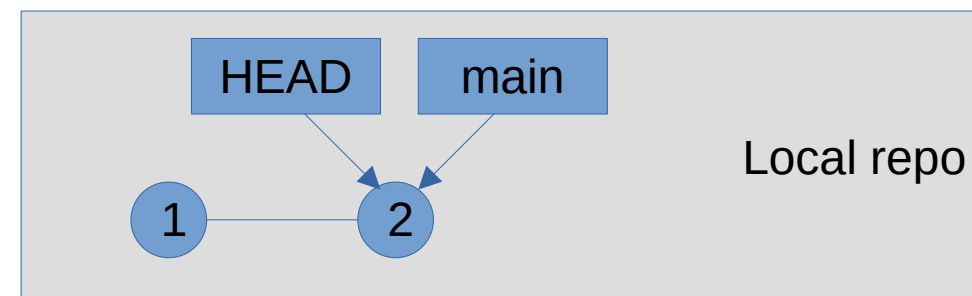


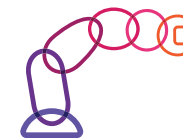
Revisions

- When we looked at the Git log, there is a long string of hexadecimal digits for each *commit*. This “*commit hash*” is a 40 character SHA-1 checksum of the snapshot of the directory tree, all files in it, the *commit* message and some other information. The hashes allows users to go back to old revisions or to compare revisions to see what has changed.
- You will also notice that the commit information has the following: “(HEAD -> master, origin/master, origin/HEAD)”. Each of these is essentially a name that points to the commit.
- “HEAD” always points at the revision that is currently checked out. “main” is the branch name on the local repo and points to a specific revision. Generally, the branch name points to the newest revision on the branch.
- The GitHub server (a *remote* repo) has its own copy of the repo that has matching names. The local repo also knows about the names so they are prefixed with the word “origin” to avoid confusion.

```
commit ce59e0503f29708901ad9c49b7422d8fb0212a46
(HEAD -> main, origin/main, origin/HEAD)
Author: Andy Blight
<57393437+AndyBlightLeeds@users.noreply.github.com>
Date: Thu Apr 23 11:09:34 2026 +0000

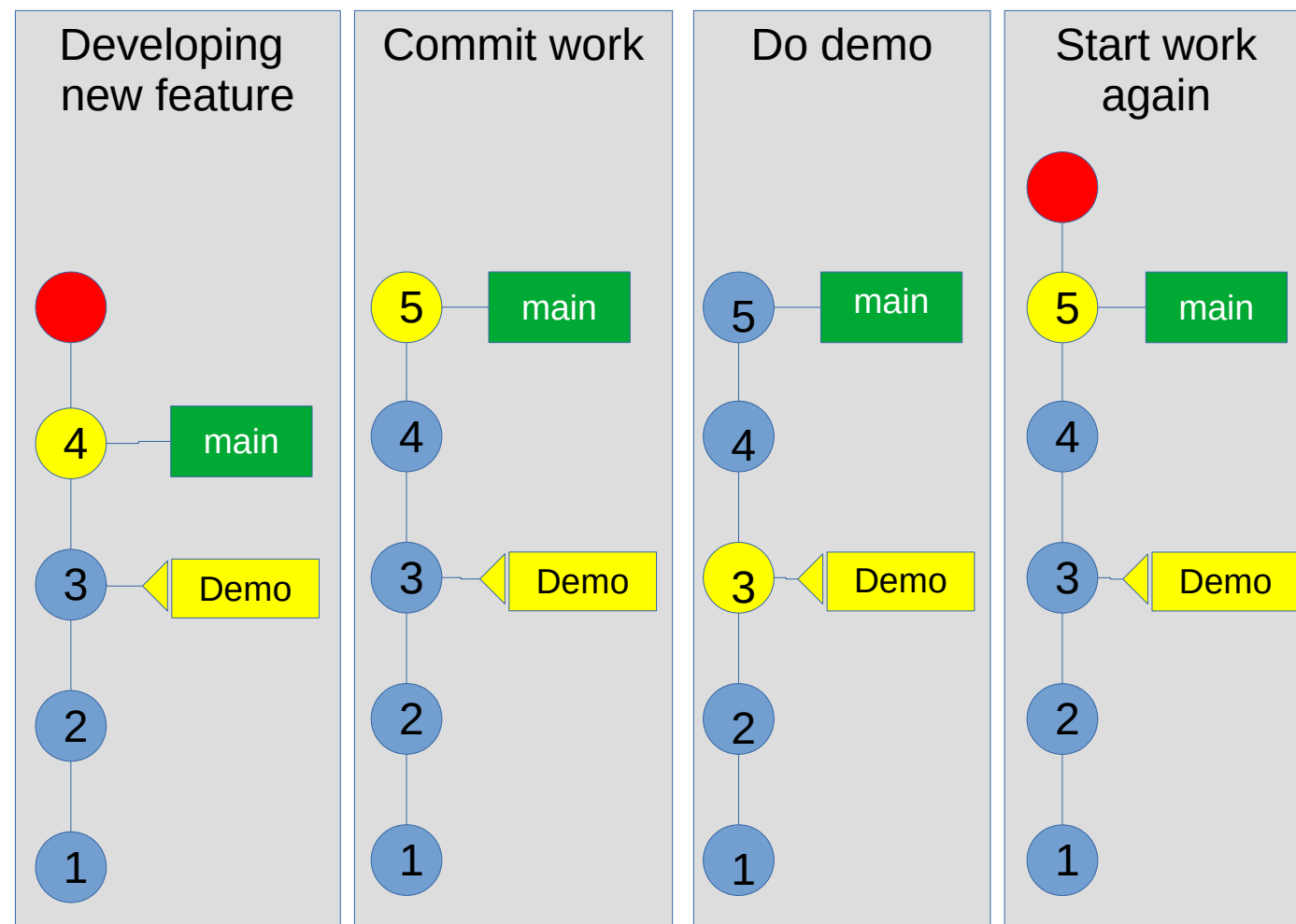
    Added diff dir
```

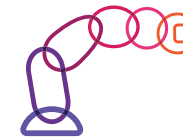




Revisions - 2

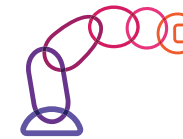
- One of the cool things about using Git is that we can go back in time (sort of!)
- For instance, you are working on adding a new feature, when your boss comes along and wants to you to show someone the demo you showed him last week. Fortunately, you added a custom label, a *tag*, to the version you showed your boss.
- You *commit* your current work.
- You *checkout* the *tag* and then do the demo.
- Afterwards, you checkout the *branch* name that you were working on and carry on with your work.





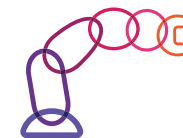
Revisions - Practical

- Lets do what we have just talked about.
 - 1) Add a *tag* to the latest revision. `git tag demo-1`
 - 2) *Commit* a new revision.
 - 3) Make some changes.
 - 4) Boss comes along and asks you for a demo. What do you do?
 - 5) Commit your work.
 - 6) *Checkout* the tagged revision. `git checkout demo-1`
 - 7) Do the demo.
 - 8) Checkout the main branch. `git checkout main`
 - 9) Start work again.



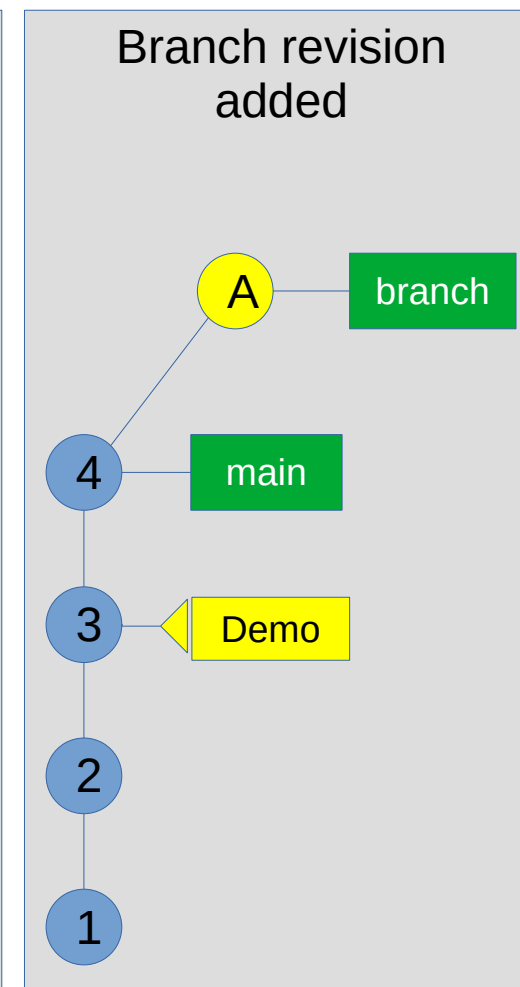
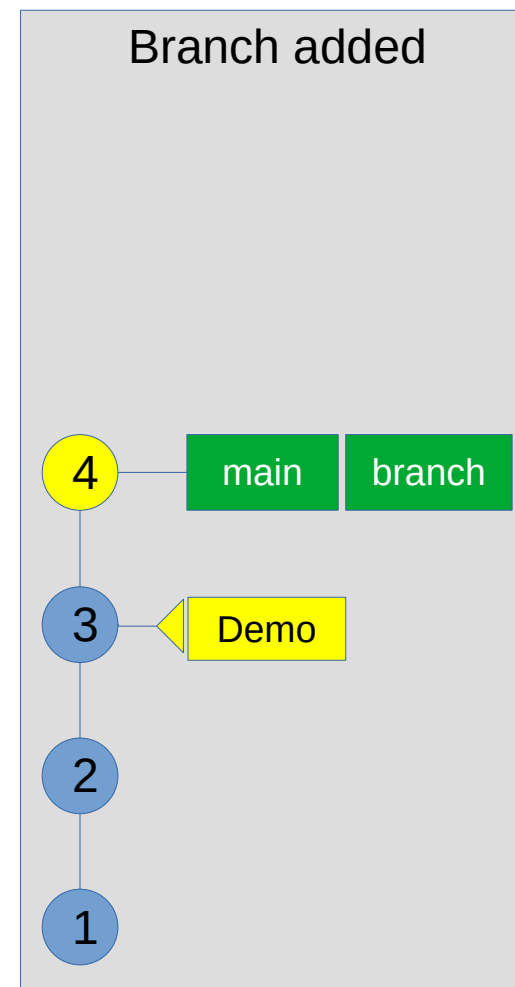
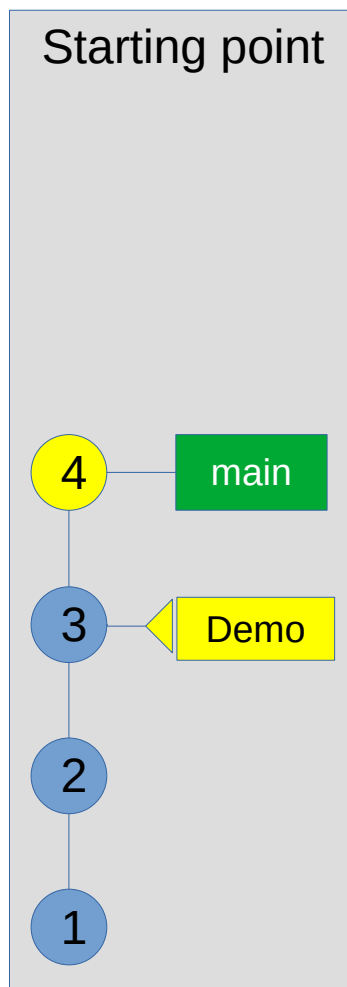
Working in small teams

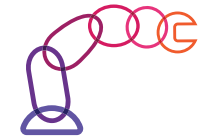
- There are several ways to collaborate using Git.
- The simplest is to let everyone in the team push directly to the `main` branch. This can be used for very small teams, say two or three people maximum but needs clear communication to avoid making mistakes. This is OK if only one person is working on the repo at a time but as soon as two people start work at the same time, things get difficult, so this is not recommended.
- A better way is for all changes have to be done on different branches. This way, several people can work on their own version of the code for as long as they like, and only when they are ready, merge their changes back to the `main` branch.
- We'll explore these methods in the next practical, but first we need to understand what branches are and how they are used.



Branches – what are they?

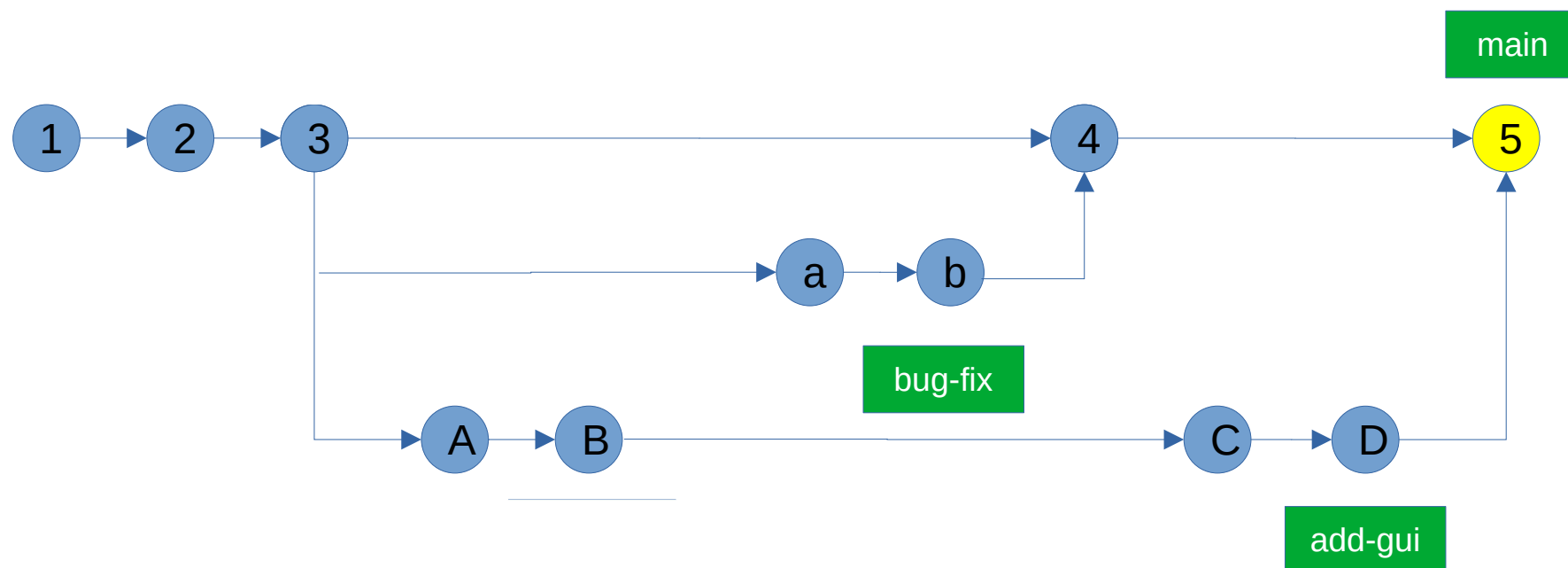
- So, what is a branch? It is essentially a label that points at a commit in a similar way to the *tag* that we used in the previous section. The *main* branch is a branch.
- When we create a new branch, all that happens is that a new label is added to the current revision.
- Once we switch to the new branch and *commit* a new revision, then the branch starts to live its own life.

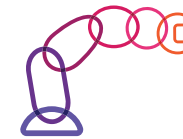




Branches - Example

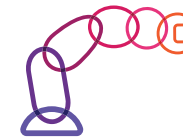
- This is an example of how one person might use two branches to work on adding a new feature and fix a bug “at the same time”. Once the changes are ready, the changes are *merged* back to the main branch.





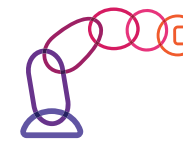
Branches - Practical

- Time to replicate the example we just saw. Open up your Codespace again and:
 - Create a branch and switch to it: **git switch -c add-gui**
 - Use **git log** to see the new branch.
 - Add a new file with some content and *commit*. Make some more changes to that file and *commit*.
 - Use **git log** to see the new commits and where the branch labels are.
 - We now need to fix a bug, so *switch* back to the *main* branch: **git switch main**
 - *Switch* to the new *bug-fix* branch: **git switch -c bug-fix**
 - Make some small changes, *commit*, make some more changes and *commit*.
 - Use **git log** to see the new commits and where the branch labels are.
 - Now we need to *merge* the changes back to the *main* branch
 - **git switch main**



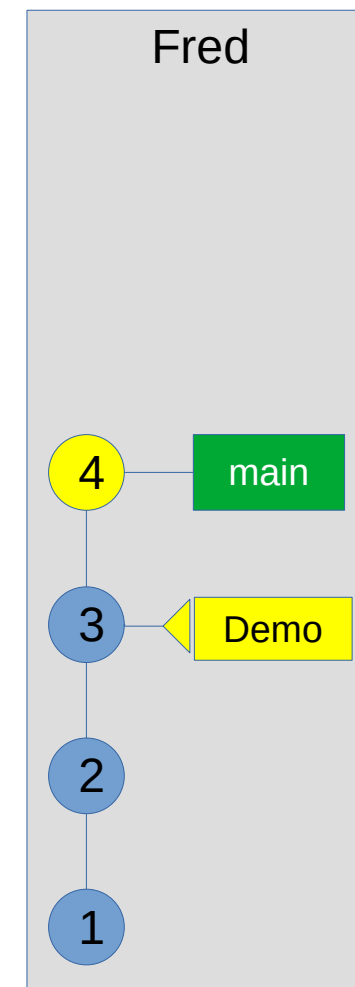
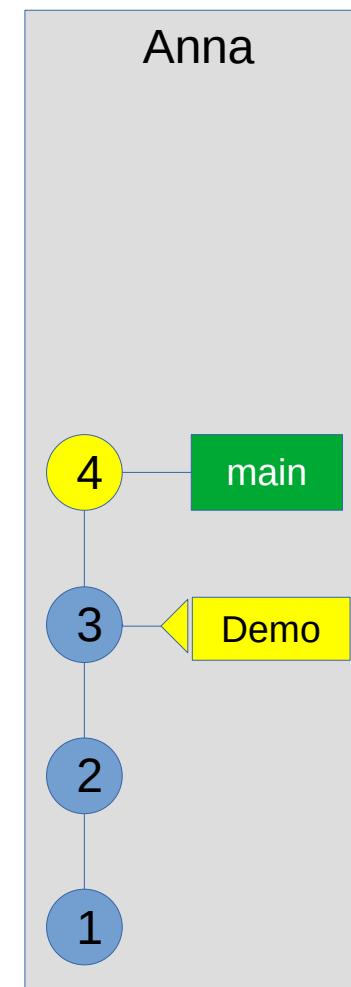
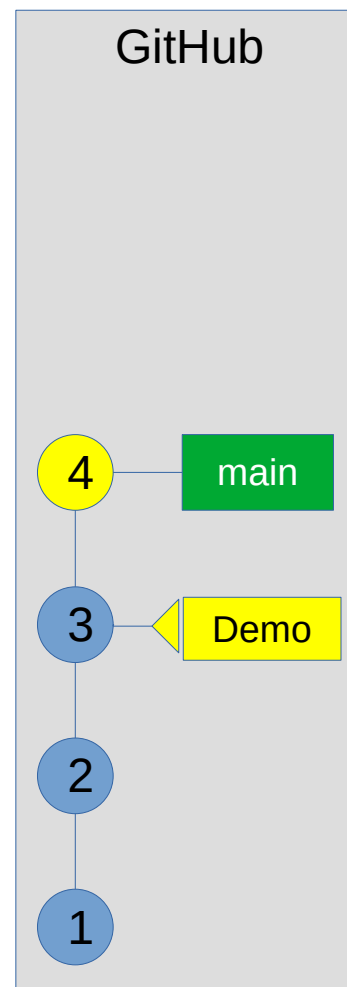
Branches – Practical 2

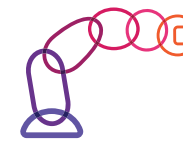
- Continued:
 - Use **git log** to see how things have changed.
 - **git merge bug-fix**
 - The bug has been fixed so we can get on with the new feature, so we *switch* back to the *add-gui* branch.
 - Use **git log** to see how things have changed.
 - Make some small changes, *commit*, make some more changes and *commit*.
 - Use **git log** to see how things have changed.
 - Now we need to *merge* the changes back to the *main* branch.
 - **git switch main**
 - **git merge add-gui**
 - Use **git log** to see how things have changed.



Working in small teams 2

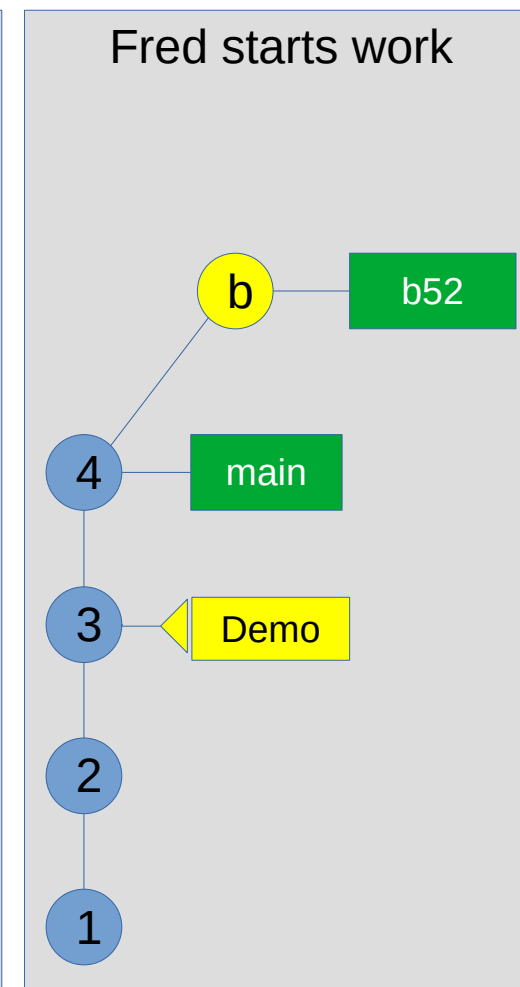
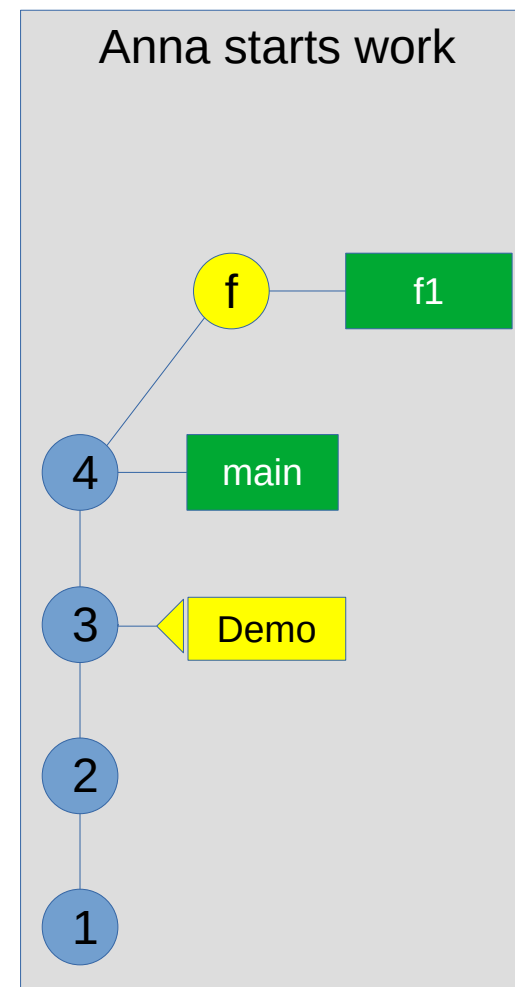
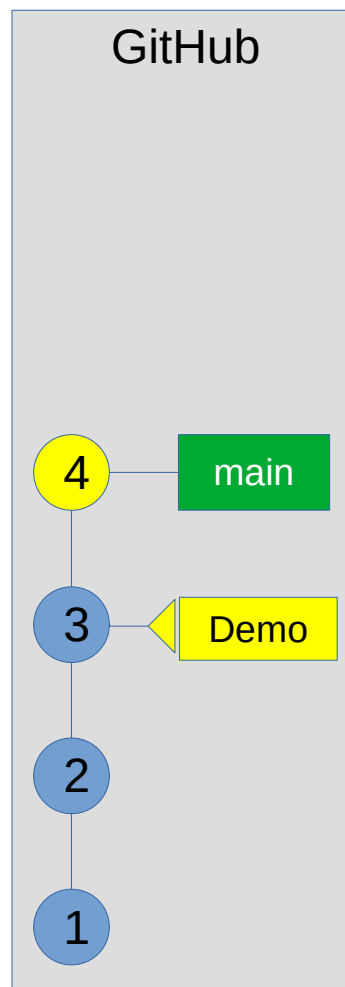
- Now that we know what branches are and how to use them, working in a small team using branches is pretty much the same as working on your own, with a few small differences.
- The first difference is that several repos are involved: the GitHub repo and one repo of each developer on their local PCs or Codespace.
- This is what each repo might look like after Anna and Fred have cloned the GitHub repo.

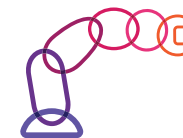




Working in small teams 3

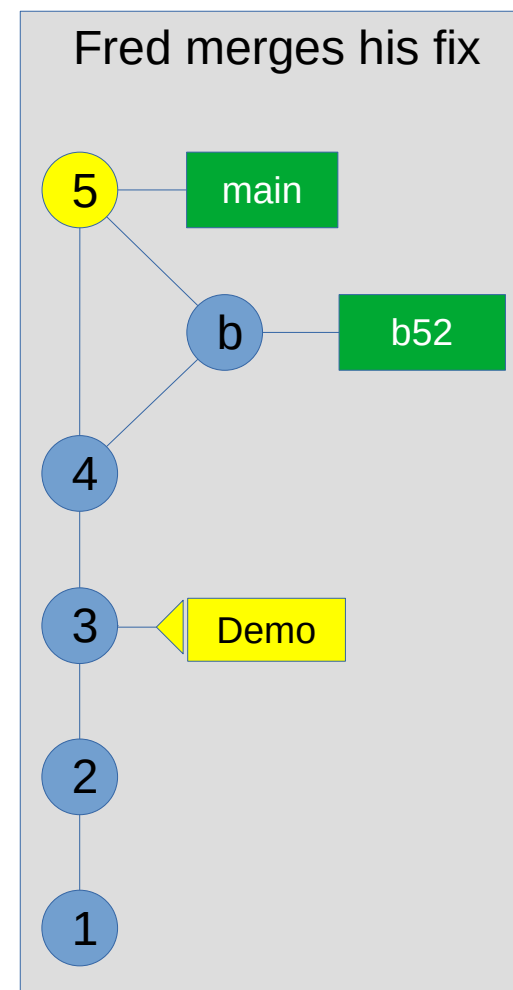
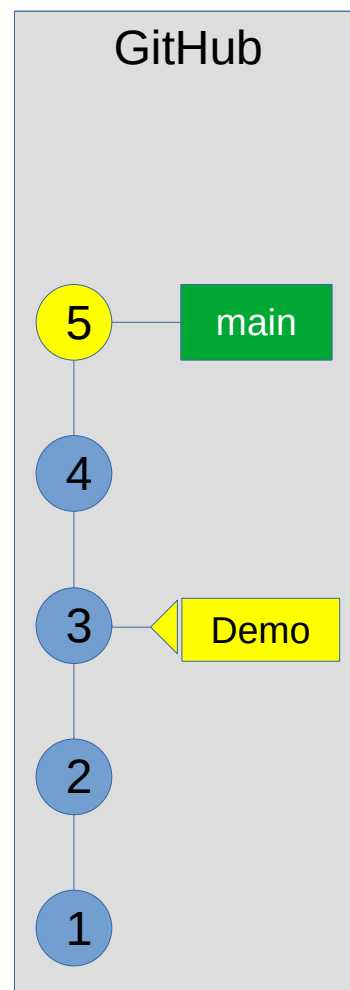
- Anna starts work a new feature, f1, so she creates a new branch with that name.
- Fred then starts work on a bug fix, b52, so he creates a new branch with that name.
- Note: branch names should be more descriptive than f1 and b52. I've kept them short as the fit on the diagram better.
- A new feature should use something like "feature/add-select-file-page". This makes it much easier for reviewers (and the developer) to know what is going on.

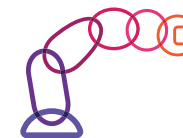




Working in small teams 4

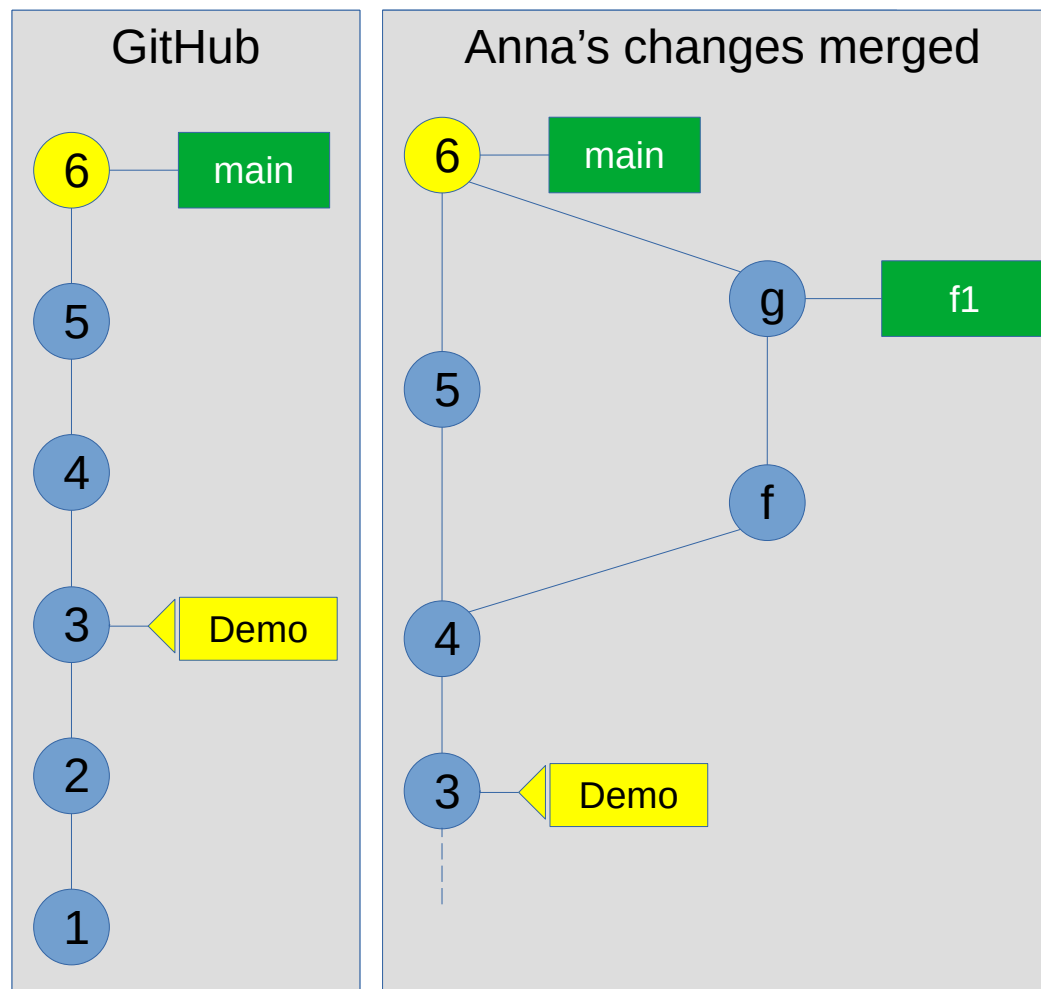
- Fred finishes his bug fix and merges the change to his local main branch.
- Then he pushes the new revision to the server.

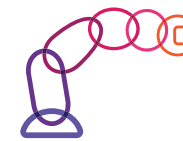




Working in small teams 5

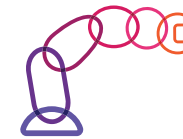
- Anna finishes the next day. She checks the GitHub repo and sees that a new change has been added to the main branch.
- Anna switches to the main branch and pulls the latest revision from the GitHub repo.
- Anna then merges her changes into the latest revision of main in her repo.
- Anna then pushes her changes to the GitHub server.





Working in small teams 6

- What can go wrong with this process? Any ideas?
- Anna forgets to pull first, merges her work and then pushes her changes to the server. This would overwrite the bug fix that Fred put in, so not good.
- In practice, this doesn't happen any more as Git has extra checks in place to prevent this happening. If Anna did this, she would receive a warning message when she tried to push and the change would stay local. Then she would have to sort out the problem locally, and push again.
- Now lets get your PC set up to use Git...

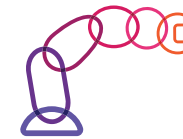


PC setup and practical

- Lets configure our PCs for using Git and GitHub.
 - See if Git is installed. Just type **git** and press enter. You should see a long help message. If not, you'll have to use the GitHub Codespace instead.
 - Ideally also install **gitk** and **meld**, although these aren't essential, they are quite useful.

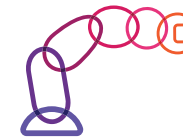
DEMO THESE ON MY PC.

- If you want the branch to be show in your prompt, then you need to clone this repo: https://github.com/andyblight/bash_scripts.git. Once cloned, follow the instructions in the README.md file to install. It takes a few minutes, so you can do this later.
- You need to configure your Git global settings to be able to commit locally. These settings are in the **~/.gitconfig** file. To add you user name and email address (the minimum needed), you run these commands (replace John Doe with your name and address):
 - **git config --global user.name "John Doe"**
 - **git config --global user.email johndoe@example.com**



PC setup – SSH keys

- To be able to push changes to the GitHub repo outside of a Codespace, there are three ways to do this:
 - 1) For repos cloned using the HTTPS option, you need to enter your user name and a personal access token (PAT). The PAT is design for use on automated systems that operate on your repos.
 - 2) For repos cloned using the SSH option, you need to create an SSH key and add it to your GitHub settings. Once set up correctly, you barely notice it.
 - 3) GitHub CLI is the third way but it is intended for power users, so we are not covering it here.
- Today, we are going to set up an SSH key as this is the best way to use GitHub. This is a bit involved, but if we can't make it work, we can do most of the work using the GitHub Codespace.
- Continued...



SSH key setup

- Please follow the instructions here:

- 1) Checking for existing SSH keys:

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/checking-for-existing-ssh-keys>

- 2) If you have an SSH key already, you can skip this next section.

- 3) Generating a new SSH key and adding it to the ssh-agent

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent?platform=linux>

- 1) In the section Generating a new SSH key, leave the passphrase blank for now.

- 2) After doing section “Adding your SSH key to the ssh-agent”, move on to step 4.

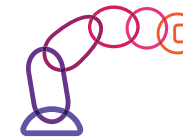
- 4) Adding a new SSH key to your GitHub account

<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account>

- 5) Testing your SSH connection

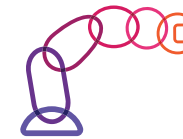
<https://docs.github.com/en/authentication/connecting-to-github-with-ssh/testing-your-ssh-connection>

Hopefully, you should see the success message.



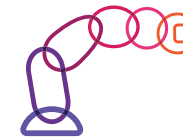
Practical – Working in small teams

- Clone this repo: **git@github.com:AndyBlightLeeds/git-github-practicals.git** for the exercises.
- Before you can push and changes to this repo, you need to be added as a collaborator. Put your GitHub user names in the chat, and I'll add you. You'll get an invitation email that you have to reply to.
- Open the repo in VSCode, open the file **working-in-small-teams/README.md** and follow the instructions.
- There are two points where I ask you to wait. I'll do a quick demo and then let you have a go. I'm hoping for a couple of problems with the merges so just let me know and we can talk them through.



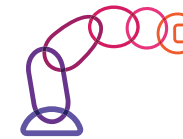
Contributing to open source projects

- So how do you contribute to an open source project using GitHub?
- Find a repo that you want to contribute to. This is often a repo that you are using but want to make a change or fix a bug.
- The process is:
 - Fork the open source repo - <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/working-with-forks/fork-a-repo>
 - Clone the forked repo.
 - Create a branch for your changes.
 - Do the work on the new branch.
 - Test and document.
 - Push the branch to your repo
 - Continued...



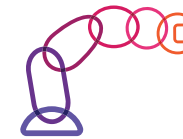
Contributing to open source projects 2

- The process continued:
 - Go to your forked repo on GitHub and submit a pull request (PR). This PR will appear on the list of PRs on the maintainer's repo.
 - The maintainer of the open source repo will review your change, ask you to fix some things or if there is nothing wrong, they will merge it to the default branch.
- One of the key benefits of this approach is that you don't have to be set up as a collaborator, so it is easy for anyone to submit PRs. The reason is that you are only working with your own fork of the repo, and only the PRs are sent to the maintainer's repo.
- Continued...



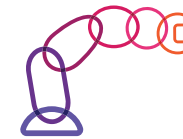
Other times to use forks

- I forked several repos when I was working on the Pipebots project. Why?
- The main reason was to prevent upstream developers changing the code we were using. If you have a robot that works, you don't want other people breaking your robot. The `main` branch could be updated but this was only done when I decided that the project was in a state where we could fix any bugs introduced.
- The second reason was to make small changes to the configuration data for your robot. To do this, a branch was created, the changes made and committed. Then whenever anyone setup a new robot, they would have the correct configuration settings.
- Finally, there were a couple of bug fixes that were needed when we built some code on a Raspberry Pi. Again, these were kept on a branch.
- To manage all the repos on a robot, I used Bash scripts and a tool called `vcstool` to clone the correct repos and branches to be used for that robot.



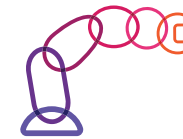
Practical – Contributing to my repo

- Practical time.
- I have setup a repo that operates like an open source repo. Link for the repo:
<https://github.com/AndyBlightLeeds/git-github-open-source-repo>
- Note to self: Do the exercise in a Codespace using your personal account.
- The README has all the instructions. Once you have pushed you changes and I'll show you how to do create a PR and the review process.



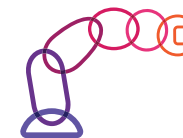
GitHub Enterprise

- GitHub Enterprise is free for academic use. It took about 6 months to get it set up for the University of Leeds as our legal department got involved.
- Benefits:
 - Setup organisations, e.g. one per school, institute or even multi-uni projects. (Show them the organisations I have setup). I am the admin for all of these organisations.
 - Have a group of university super admins that over see all repos, so if access is lost, it can be accessed again.
 - Data security: Backups that you don't have to think about.
 - 25,000 licenses. We've used just over a 1000 so far.
 - Can transfer repos from one organisation to another.
 - Classroom options available as well.
- Let's take a look at the setup I look after at Leeds.



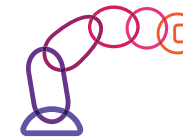
Final tricks and tips

- There is one file that is useful to know about called **.gitignore** It tells Git to ignore files that match the patterns that are in the **.gitignore** file. For example:
 - ***.tmp** will ignore all files ending in **.tmp**
 - ****/.vscode** will ignore all directories anywhere in the tree that have the name **.vscode**
 - This is really useful when you are building C or C++ projects as the compiler generates many intermediate files that you do not want to keep.
 - I'll give you a quick demo...



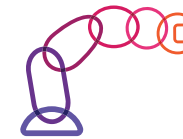
Anything else?

- Questions, thoughts, any feedback.
- Fill out the survey: https://www.smartsurvey.co.uk/s/UK_RAS_STEPS_activities_feedback/



References

- There is lots of information on Git and GitHub on the Internet. Here are some of the references I used when creating this workshop.
 - <https://git-scm.com/> This is my go to reference for using Git. It has lots of detail, so might be confusing for beginners.
 - <https://git-scm.com/book/en/v2/Getting-Started-A-Short-History-of-Git>
 - <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>
 - <https://arctraining.github.io/git/> I used this as a starting point for this course. Might be useful as a reference.
 - Continued...



References

- GitHub docs
 - [Everything to do with GitHub is documented here](#)
 - [GitHub work flow \(PRs and reviews\)](#)
 - [Fork a repo](#)
 - [Create a personal access token](#)
 - [Create an SSH key](#)
 -